

UNIT-5

PHP and MYSQL

PHP Basics – Arrays - Functions– Form Handling - File handling – Date and Time – MYSQL Basics - PHP and MYSQL Database Connectivity- Session - Cookies, E-mail – PHP and XML –PHP and AJAX.

By

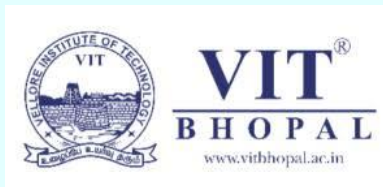
Dr. Adyasha Sahu

Assistant Professor

School of Computing Science Engineering and

Artificial Intelligence (SCAI)

VIT Bhopal University



PHP Introduction

- PHP was created by **Rasmus Lerdorf in 1994** but appeared in the market in 1995.
- PHP is a server-side scripting language
- PHP scripts are executed on the server
- PHP supports many databases (MySQL, Informix, Oracle, Sybase, Solid, PostgreSQL, Generic ODBC, etc.)
- PHP is open source software
- PHP is free to download and use

PHP Introduction

- PHP runs on different platforms (Windows, Linux, Unix, etc.)
- PHP is compatible with almost all servers used today (Apache, IIS, etc.)
- PHP is FREE to download from the official PHP resource: www.php.net
- PHP is easy to learn and runs efficiently on the server side.
- PHP supports several protocols such as HTTP, POP3, SNMP, LDAP, IMAP, and many more.

PHP Introduction

- Instead of lots of commands to output HTML (as seen in C or Perl), PHP pages contain HTML with embedded code that does "something" (like in the next slide, it outputs "Hi, I'm a PHP script!").
- The PHP code is enclosed in special start and end processing instructions `<?php` and `?>` that allow you to jump into and out of "PHP mode."
- PHP can handle the forms, such as - collect the data from users using forms, save it into the database, and return useful information to the user. For example - Registration form.

PHP Features

- Performance
- Open Source
- Embedded
- Control
- Loosely Typed Language
- Security
- Helpful PHP Community
- Web Server Support
- Error Reporting
- Platform Independent
- Familiarity with syntax

Install PHP

- WAMP- Windows-<https://www.wampserver.com/en/>
- LAMP- Linux-
<http://csg.sph.umich.edu/abecasis/LAMP/download/>
- MAMP- Mac-<https://www.mamp.info/en/downloads/>
- XAMPP- Apache, MySQL, PHP, Perl
-<https://www.apachefriends.org/download.html>

PHP Introduction

```
<html>  
<body>  
<?php  
echo "<h2>Hello First PHP</h2>";  
?>  
</body>  
</html>
```

PHP Hello World

- This program is extremely simple and you really did not need to use PHP to create a page like this. All it does is display: Hello World using the PHP **echo()** statement.
- Think of this as a normal HTML file which happens to have a set of special tags available to you that do a lot of interesting things.

PHP Comments

- In PHP, we use `//` to make a single-line comment or `/*` and `*/` to make a large comment block.

```
<html>
<body>

<?php
//This is a comment

/*
This is
a comment
block
*/
?>

</body>
</html>
```

PHP Variables

- In PHP, a variable starts with the \$ sign, followed by the name of the variable:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<?php
```

```
$txt = "University";
```

```
echo "VIT Bhopal $txt!";
```

```
?>
```

```
</body>
```

```
</html>
```

PHP Math

- The `min()` and `max()` functions can be used to find the lowest or highest value in a list of arguments:

```
<?php  
echo(min(0, 150, 30, 20, -8, -200));  
// returns -200  
echo(max(0, 150, 30, 20, -8, -  
200)); // returns 150  
?>
```

PHP Constant Arrays

- An Array constant using the define() function

```
<?php  
define("Branch", [  
    "AI",  
    "ML",  
    "CI"  
]);  
echo cars[0];  
?>
```

PHP Arrays

- **Indexed arrays** - Arrays with a numeric index
- **Associative arrays** - Arrays with named keys
- **Multidimensional arrays** - Arrays containing one or more arrays

```
<!DOCTYPE html>
<html>
<body>
<?php
$scars = array("Volvo",
"BMW", "Toyota");
echo "I like " . $scars[0] .
", " . $scars[1] . " and " .
$scars[2] . ".";
?>
</body>
</html>
```

PHP Arrays

- **Indexed arrays** –

```
$cars[0] = "AI";
```

```
$cars[1] = "ML";
```

```
$cars[2] = "CI";
```

- **Associative arrays** –

```
$age['Abhinav'] = "35";
```

```
$age['Rishav'] = "37";
```

```
$age['Shanu'] = "43";
```

- **Multidimensional arrays** –

```
$cars = array (  
    array("Volvo",22,  
        18),  
    array("BMW",15,1  
        3),  
    array("Saab",5,2),  
        array("Land  
        Rover",17,15)  
);
```

What is a PHP File?

1. PHP files can contain text, HTML, CSS, JavaScript, and PHP code
2. PHP code are executed on the server, and the result is returned to the browser as plain HTML
3. PHP files have extension ".php"

What Can PHP Do?

PHP can generate dynamic page content

PHP can create, open, read, write, delete, and close files on the server

PHP can collect form data

PHP can send and receive cookies

PHP can add, delete, modify data in your database

PHP can encrypt data

Why PHP?

PHP runs on various platforms (Windows, Linux, Unix, Mac OS X, etc.)

PHP is compatible with almost all servers used today (Apache, IIS, etc.)

PHP supports a wide range of databases

PHP is free. Download it from the official

PHP resource: www.php.net

PHP is easy to learn and runs efficiently on the server side

To start using PHP, you can:

Find a web host with PHP and MySQL support
Install a web server on your own PC, and then install
PHP and MySQL

Use a Web Host with PHP Support

If your server has activated support for PHP you do not need to do anything.

Just create some .php files, place them in your web directory, and the server will automatically parse them for you.

You do not need to compile anything or install any extra tools.

Because PHP is free, most web hosts offer PHP support.

Set Up PHP on Your Own PC

However, if your server does not support PHP, you must:

- install a web server
- install PHP
- install a database, such as MySQL

Data Types

Variables can store data of different types, and different data types can do different things.

PHP supports the following data types:

String

Integer

Float (floating point numbers - also called double)

Boolean

Array

Object

NULL

Resource

PHP String

A string is a sequence of characters, like "Hello world!".

A string can be any text inside quotes. You can use single or double quotes:

Example

```
<html>
<body>
<?php
$x = "Hello world!";
$y = 'Hello world!'; echo $x;
echo "<br>"; echo $y;
?>
</body>
</html>
```

OUTPUT:

Hello world! Hello world!

String Functions

Get The Length of a String

The PHP **strlen()** function returns the length of a string.

The example below returns the length of the string "Hello world!"

```
<html>
<body>
<?php
echo strlen("Hello world!"); ?>
</body>
</html>
```

str_word_count()

strrev()

str_replace()

OUTPUT:

12

PHP Integer

An integer data type is a non-decimal number between -2,147,483,648 and 2,147,483,647.

Rules for integers:

- An integer must have at least one digit

- An integer must not have a decimal point

- An integer can be either positive or negative

- Integers can be specified in three formats: decimal (10-based), hexadecimal (16- based - prefixed with 0x) or octal (8-based - prefixed with 0)

- In the following example \$x is an integer.

The PHP **var_dump()** function returns the data type and value:

```
<html>  
<body>  
<?php  
$x = 5985;  
var_dump($x);  
?>  
</body>  
</html>
```

OUTPUT:
int(5985)

PHP Float

A float (floating point number) is a number with a decimal point or a number in exponential form.

In the following example \$x is a float. The PHP var_dump() function returns the data type and value:

PHP Boolean

A Boolean represents two possible states: TRUE or FALSE.

```
$x = true;
```

```
$y = false;
```

PHP Array

An array stores multiple values in one single variable:

An array is a special variable, which can hold more than one value at a time.

If you have a list of items (a list of car names, for example), storing the cars in single variables could look like this:

```
$cars1 = "Volvo";
```

```
$cars2 = "BMW";
```

```
$cars3 = "Toyota";
```

However, what if you want to loop through the cars and find a specific one? And what if you had not 3 cars, but 300?

The solution is to create an array!

An array can hold many values under a single name, and you can access the values by referring to an index number.

```
<html>
<body>
<?php
$cars = array("Volvo", "BMW", "Toyota");
echo "I like " . $cars[0] . ", " . $cars[1] . " and
" . $cars[2] . ".";
?>
</body>
</html>
```

OUTPUT:

I like Volvo, BMW and Toyota.

Create an Array in PHP

In PHP, the **array()** function is used to create an array:

```
array();
```

In PHP, there are three types of arrays:

Indexed arrays - Arrays with a numeric index

Associative arrays - Arrays with named keys

Multidimensional arrays - Arrays containing one or more arrays

Variables

Variables are "containers" for storing information.

Creating (Declaring) PHP Variables

In PHP, a variable starts with the \$ sign, followed by the name of the variable:

- `rsort()` - sort arrays in descending order
- `asort()` - sort associative arrays in ascending order, according to the value
- `ksort()` - sort associative arrays in ascending order, according to the key
- `arsort()` - sort associative arrays in descending order, according to the value
- `krsort()` - sort associative arrays in descending order, according to the key

Rules for PHP variables:

A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume).

A variable starts with the \$ sign, followed by the name of the variable

A variable name must start with a letter or the underscore character

A variable name cannot start with a number

A variable name can only contain alpha-numeric characters and underscores (A- z, 0-9, and _)

Variable names are case-sensitive (\$age and \$AGE are two different variables)

PHP Variables Scope

In PHP, variables can be declared anywhere in the script.

The scope of a variable is the part of the script where the variable can be referenced/used.

PHP has three different variable scopes:

local

global

static

Global and Local Scope

A variable declared outside a function has a GLOBAL SCOPE and can only be accessed outside a function:


```
<html>
<body>
<?php
$x = 5; // global scope function myTest() {
// using x inside this function will generate an error echo
"<p>Variable x inside function is: $x</p>";
}
myTest();
echo "<p>Variable x outside function is:
$x</p>"; ?>
</body>
</html>
```

OUTPUT:

Variable x inside function is:

Variable x outside function is: 5

```
<html>
<body>
<?php
function myTest() {
    $x = 5; // local scope
    echo "<p>Variable x inside function is: $x</p>";
}
myTest();
// using x outside the function will generate an error echo
"<p>Variable x outside function is:
    $x</p>"; ?>
</body>
</html>
```

OUTPUT:

Variable x inside function is: 5 Variable x outside function is.

The global Keyword

```
<html>
<body>
<?php
$x = 5;
$y = 10
unction myTest() { global $x, $y; $y
= $x + $y;
}
myTest(); // run function
echo $y; // output the new value for variable
$y ?>
</body>
</html>
```

The static Keyword

Normally, when a function is completed/executed, all of its variables are deleted. However, sometimes we want a local variable NOT to be deleted. We need it for a further job.

```
<html>
<body>
<?php
function myTest() { static $x = 0; echo $x;
$x++;
}
myTest(); echo "<br>"; myTest(); echo "<br>"; myTest();
?>
</body>
</html>
```

Output:

0
1
2

echo and print Statements

In PHP there are two basic ways to get output: `echo` and `print`. In this tutorial we use `echo` (and `print`) in almost every example. So, this chapter contains a little more info about those two output statements.

The differences are small: `echo` has no return value while `print` has a return value of 1 so it can be used in expressions. `echo` can take multiple parameters (although such usage is rare) while `print` can take one argument. `echo` is marginally faster than `print`.

PHP Object

An object is a data type which stores data and information on how to process that data. In PHP, an object must be explicitly declared.

First we must declare a class of object. For this, we use the class keyword. A class is a structure that can contain properties and methods:

PHP NULL Value

Null is a special data type which can have only one value: NULL.

A variable of data type NULL is a variable that has no value assigned to it.

If a variable is created without a value, it is automatically assigned a value of NULL.

Variables can also be emptied by setting the value to NULL:

PHP Operators

Operators are used to perform operations on variables and values.

PHP divides the operators in the following groups:

Arithmetic operators

Assignment operators

Comparison operators

Increment/Decrement operators

Logical operators

String operators

Array operators

	==	Equality
	===	Identity
++\$x	!=	Inequality
\$x++	<>	Inequality
--\$x	!==	Non-identity
\$x--		

- # PHP Conditional Statements

- **if statement** - executes some code if one condition is true
- **if...else statement** - executes some code if a condition is true and another code if that condition is false
- **if...elseif...else statement** - executes different codes for more than two conditions
- **switch statement** - selects one of many blocks of code to be executed

if statement - executes some code if one condition is true

```
if (condition) {  
    code to be executed if condition is true;  
}
```

The example below will output "Have a good day!" if the current time (HOUR) is less than 20:

```
<html>  
<body>  
<?php  
$t = date("H");  
if ($t < "20") {  
    echo "Have a good day!";  
}  
?>  
</body>  
</html>
```

OUTPUT:

Have a good day!

The if...else Statement

Syntax

```
if (condition) {  
  code to be executed if condition is true;  
} else {  
  code to be executed if condition is false;  
}
```

The example below will output "Have a good day!" if the current time is less than 20, and "Have a good night!" otherwise:

```
<html>
<body>
<?php
$t = date("H");
if ($t < "20") {
echo "Have a good day!";
} else {
echo "Have a good night!";
}
?>
</body>
</html>
```

OUTPUT:

Have a good day!

The if...elseif...else Statement

```
if (condition) {  
    code to be executed if this condition is true;  
} elseif (condition) {  
    code to be executed if this condition is true;  
} else {  
    code to be executed if all conditions are  
    false;  
}
```

```
<html>
<body>
<?php
$t = date("H");
echo "<p>The hour (of the server) is " . $t;

echo ", and will give the

following message:</p>";

if ($t < "10") {

    echo "Have a good morning!";

} elseif ($t < "20") {
    echo "Have a good day!";
} else {
    echo "Have a good night!";
}
?>
</body>
</html>
```

OUTPUT:

Have a good morning!

The switch Statement

The switch statement is used to perform different actions based on different conditions.

```
<html>
<body>
<?php
$favcolor = "red"; switch ($favcolor) {
case "red":
echo "Your favorite color is red!"; break;
case "blue":
echo "Your favorite color is blue!"; break;
case "green":
echo "Your favorite color is green!"; break;
default:
echo "Your favorite color is neither red, blue, nor green!";
}
?>
</body>
</html>
```

OUTPUT:

Your favorite color is red!

PHP Loops

- **while**- loops through a block of code as long as the specified condition is true
- **do...while** - loops through a block of code once, and then repeats the loop as long as the specified condition is true
- **for**- loops through a block of code a specified number of times
 - **foreach**- loops through a block of code for each element in an array

PHP Loops

The PHP while Loop

```
<html>
<body>
<?php
$x = 1;
while($x <= 5) {
echo "The number is: $x
<br>"; $x++;
}
?>
</body>
</html>
```

OUTPUT:

*The number is: 1 The number is: 2 The number is: 3 The
number is: 4 The number is: 5*

The PHP do...while Loop

```
<html>  
<body>  
<?php  
$x = 1; do {  
echo "The number is: $x <br>";  
$x++;  
} while ($x <= 5); ?>  
</body>  
</html>
```

OUTPUT:

The number is: 1 The number is: 2 The number is: 3 The number is: 4 The number is: 5

The PHP for Loop

The **for** loop is used when you know in advance how many times the script should run.

Syntax

```
for (init counter; test counter; increment counter) {  
code to be executed;  
}
```

Parameters:

init counter: Initialize the loop counter value

test counter: Evaluated for each loop iteration. If it evaluates to TRUE, the loop continues. If it evaluates to FALSE, the loop ends.

increment counter: Increases the loop counter value

```
<html>
<body>
<?php
for ($x = 0; $x <= 10; $x++) { echo "The number is: $x
<br>";
}
?>
</body>
</html>
```

OUTPUT:

***The number is: 0 The number is: 1 The number is: 2 The
number is: 3 The number is: 4 The number is: 5 The
number is: 6 The number is: 7 The number is: 8 The
number is: 9 The number is: 10***

The PHP foreach Loop

The foreach loop works only on arrays, and is used to loop through each key/value pair in an array.

Syntax

```
foreach ($array as $value)
```

```
{  
code to be executed;  
}
```

For every loop iteration, the value of the current array element is assigned to \$value and the array pointer is moved by one, until it reaches the last array element.

The following example demonstrates a loop that will output the values of the given array (\$colors):

```
<html>
<body>
<?php
$colors = array("red", "green", "blue", "yellow");

foreach ($colors as $value)
{ echo "$value <br>";
}
?>
</body>
</html>
```

OUTPUT:

red
green
blue
yellow

Functions

- The real power of PHP comes from its functions; it has more than 1000 built-in functions.

PHP User Defined Functions

- Besides the built-in PHP functions, we can create our own functions.
- A function is a block of statements that can be used repeatedly in a program.
- A function will not execute immediately when a page loads.
- A function will be executed by a call to the function.

Create a User Defined Function in PHP

A user-defined function declaration starts with the word function:

Syntax

```
function functionName() {  
    code to be executed;  
}
```



```
<html>  
<body>  
<?php  
function writeMsg()  
echo "Hello world!";  
}  
writeMsg();  
?>  
</body>  
</html>
```

OUTPUT:
Hello world!

PHP Function Arguments

- Information can be passed to functions through arguments. An argument is just like a variable.
- Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

```
<html>
<body>
<?php
function familyName($fname) { echo "$fname Refsnes.<br>";
}
familyName("Jani"); familyName("Hege"); familyName("Stale");
familyName("Kai Jim"); familyName("Borge");
?>
</body>
</html>
```

PHP Return Value

```
<html>
<body>
<?php
function sum($x, $y) {
    $z = $x + $y; return $z;
}
echo "5 + 10 = " . sum(5,10) . "<br>";
echo "7 + 13 = " . sum(7,13) . "<br>"; echo "2 + 4 = " . sum(2,4);
?>
</body>
</html>
```

OUTPUT:

5 + 10 = 15

File handling

It is an important part of any web application. You often need to open and process a file for different tasks.

PHP Manipulating Files

PHP has several functions for creating, reading, uploading, and editing files.

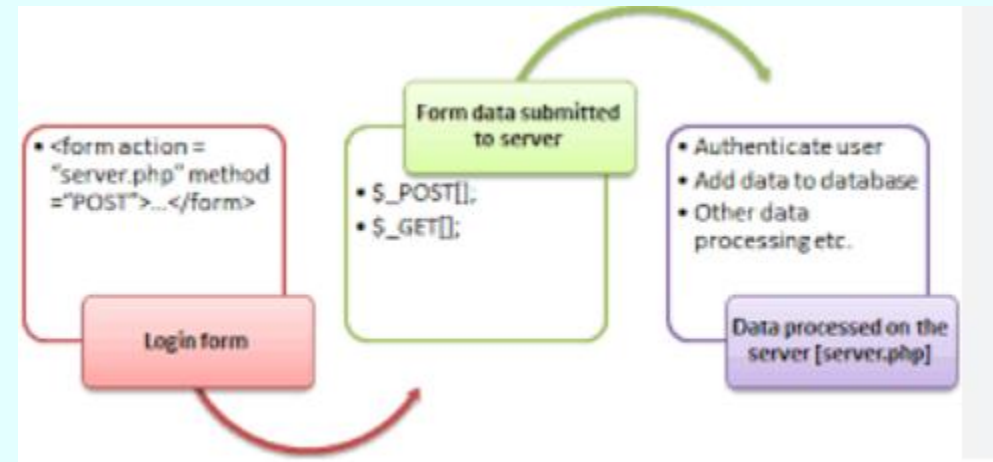
1. The **readfile()** function is useful if all you want to do is open up a file and read its contents.
2. A better method to open files is with the **fopen()** function. This function gives you more options than the readfile() function.
3. PHP Read Single Character - **fgetc()**

```
<?php  
$myfile = fopen("any.txt", "r") or  
die("Unable to open file!"); echo  
fread($myfile, filesize("any.txt"));  
fclose($myfile);  
?>
```

Supported modes are:

r	- Read Only	r+	- Read & Write
w	- Write Only	w+	- Read & Write
a	- Write Only(Append).		

PHP Form Handling



The PHP superglobals `$_GET` and `$_POST` are used to collect form-data.

```
<html>
```

```
<body>
```

```
<form action="welcome.php" method="post">
```

```
Name: <input type="text" name="name"><br>
```

```
E-mail: <input type="text" name="email"><br>
```

```
<input type="submit">
```

```
</form>
```

```
</body>
```

```
</html>
```

GET may be used for sending non-sensitive data.

PHP Date and Time

```
date(format,timestamp)
```

```
|
```

```
|
```

```
|
```

```
<?php
```

```
echo "Today is " . date("Y/m/d") . "<br>";
```

```
echo "Today is " . date("Y.m.d") . "<br>";
```

```
echo "Today is " . date("Y-m-d") . "<br>";
```

```
echo "Today is " . date("l");
```

```
?>
```

O/P= |

Today is 2025/09/21

Today is 2025.09.21

Today is 2025-09-21

Today is Sunday

MYSQL Basics

- world's most popular open source database because of its consistent **fast performance, high reliability and ease of use**
- Open Source License:- free
 - GNU General Public License
 - Free to modify and distribute but all modification must be available in source code format
- Commercial:- not free
 - Fully paid up professional support
- **used by Google, Facebook Nokia, YouTube, Yahoo!, Alcatel-Lucent, Zappos.com, etc.**

MYSQL Basics

- **Database runs as a server**
 - Attaches to either a default port or an administrator specified port
- **Clients connect to database**
 - For secure systems
 - authenticated connections
 - usernames and passwords
- **Clients make queries on the database**
 - Retrieve content
 - Insert content
- **SQL (Structured Query Language)** is the language used to insert and retrieve content

MYSQL Basics

- **MySQL can be controlled through a simple command-line interface; however, we can use phpMyAdmin as an interface to MySQL.**
- **phpMyAdmin is a very powerful tool; it provides a large number of facilities for customising a database management system.**

- The data in a MySQL database are stored in tables. A table is a collection of related data, and it consists of columns and rows.
- Databases are useful for storing information categorically.
- With PHP, you can connect to and manipulate databases.
- MySQL is the most popular database system used with PHP.

PHP Connect to MySQL

PHP 5 and later can work with a MySQL database using:

- **MySQLi extension** (the "i" stands for improved)
- **PDO (PHP Data Objects)**

Earlier versions of PHP used the MySQL extension. However, this extension was deprecated in 2012.

What should be used MySQLi or PDO?

Anyone can be used. Both MySQLi and PDO have their advantages:

- PDO will work on 12 different database systems, whereas MySQLi will only work with MySQL databases.
- So, if you have to switch your project to use another database, PDO makes the process easy. You only have to change the connection string and a few queries. With MySQLi, you will need to rewrite the entire code - queries included.
- Both are object-oriented, but MySQLi also offers a procedural API.
- Both support Prepared Statements. Prepared Statements protect from SQL injection, and are very important for web application security.

MySQLi Installation

For Linux and Windows: The MySQLi extension is automatically installed in most cases, when php5 mysql package is installed.

For installation details, go to: <http://php.net/manual/en/mysqli.installation.php>

PDO Installation

For installation details, go to: <http://php.net/manual/en/pdo.installation.php>

There are three ways of working with PHP and MySQL:

- MySQLi (object-oriented)
- MySQLi (procedural)
- PDO

Open a Connection to MySQL

Before we can access data in the MySQL database, we need to be able to connect to the server:

Example (MySQLi Object-Oriented)

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";

// Create connection
$conn = new mysqli($servername, $username, $password);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
echo "Connected successfully";
?>
```

Open a Connection to MySQL

Example (MySQLi Procedural)

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";

// Create connection
$conn = mysqli_connect($servername, $username, $password);

// Check connection
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
echo "Connected successfully";
?>
```


Open a Connection to MySQL

Example (POD)

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";

try {
    $conn = new PDO("mysql:host=$servername;dbname=myDB", $username, $password);
    // set the PDO error mode to exception
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    echo "Connected successfully";
} catch(PDOException $e) {
    echo "Connection failed: " . $e->getMessage();
}
?>
```

Open a Connection to MySQL

POD:

- In the PDO example above we have also specified a database (myDB). PDO require a valid database to connect to. If no database is specified, an exception is thrown.
- A great benefit of PDO is that it has an exception class to handle any problems that may occur in our database queries. If an exception is thrown within the `try{ }` block, the script stops executing and flows directly to the first `catch(){ }` block.

Close the Connection

- The connection will be closed automatically when the script ends. To close the connection before, use the following:

MySQLi Object-Oriented:

```
$conn->close();
```

MySQLi Procedural:

```
mysqli_close($conn);
```

PDO:

```
$conn = null;
```

PHP and XML

What is XML?

- The XML language is a way to structure data for sharing across websites.
- Several web technologies like RSS Feeds and Podcasts are written in XML.
- XML is easy to create. It looks a lot like HTML, except that you make up your own tags.

What is an XML Parser?

- To read and update, create and manipulate an XML document, you will need an XML parser.
- In PHP there are two major types of XML parsers:
 - Tree-Based Parsers
 - Event-Based Parsers

PHP and XML

Tree-Based Parsers

- Tree-based parsers holds the entire document in Memory and transforms the XML document into a Tree structure. It analyzes the whole document, and provides access to the Tree elements (DOM).
- This type of parser is a better option for smaller XML documents, but not for large XML document as it causes major performance issues.
- Example of tree-based parsers:
 - SimpleXML
 - DOM

PHP and XML

Event-Based Parsers:

- Event-based parsers do not hold the entire document in Memory, instead, they read in one node at a time and allow you to interact with in real time. Once you move onto the next node, the old one is thrown away.
- This type of parser is well suited for large XML documents. It parses faster and consumes less memory.
- Example of event-based parsers:
 - XMLReader
 - XML Expat Parser

PHP and AJAX

- AJAX is used to create more interactive applications.

AJAX PHP Example

- The following example will demonstrate how a web page can communicate with a web server while a user type characters in an input field:

- Example:

Start typing a name in the input field below:

First name:

Suggestions:

- In the example above, when a user types a character in the input field, a function called `showHint()` is executed.
- The function is triggered by the onkeyup event.

PHP and AJAX

- **HTML Code:**

```
<html>
<head>
<script>
function showHint(str) {
    if (str.length == 0) {
        document.getElementById("txtHint").innerHTML = "";
        return;
    } else {
        var xmlhttp = new XMLHttpRequest();
        xmlhttp.onreadystatechange = function() {
            if (this.readyState == 4 && this.status == 200) {
                document.getElementById("txtHint").innerHTML = this.responseText;
            }
        };
        xmlhttp.open("GET", "gethint.php?q=" + str, true);
        xmlhttp.send();
    }
}
</script>
</head>
<body>
```


PHP and AJAX

- **HTML Code:**

```
<p><b>Start typing a name in the input field below:</b></p>
<form action="">
  <label for="fname">First name:</label>
  <input type="text" id="fname" name="fname" onkeyup="showHint(this.value)">
</form>
<p>Suggestions: <span id="txtHint"></span></p>
</body>
</html>
```

Code explanation:

- First, check if the input field is empty (`str.length == 0`). If it is, clear the content of the `txtHint` placeholder and exit the function.
- However, if the input field is not empty, do the following:
- Create an **XMLHttpRequest** object
- Create the function to be executed when the server response is ready
- Send the request off to a PHP file (`gethint.php`) on the server
- Notice that **q** parameter is added to the url (`gethint.php?q=" + str`)
- And the **str** variable holds the content of the input field

PHP and AJAX

- **HTML Code:**

```
<p><b>Start typing a name in the input field below:</b></p>
<form action="">
  <label for="fname">First name:</label>
  <input type="text" id="fname" name="fname" onkeyup="showHint(this.value)">
</form>
<p>Suggestions: <span id="txtHint"></span></p>
</body>
</html>
```

Code explanation:

- First, check if the input field is empty (`str.length == 0`). If it is, clear the content of the `txtHint` placeholder and exit the function.
- However, if the input field is not empty, do the following:
- Create an **XMLHttpRequest** object
- Create the function to be executed when the server response is ready
- Send the request off to a PHP file (`gethint.php`) on the server
- Notice that **q** parameter is added to the url (`gethint.php?q=" + str`)
- And the **str** variable holds the content of the input field

PHP and AJAX

- The PHP File - "gethint.php"
- The PHP file checks an array of names, and returns the corresponding name(s) to the browser:

```
// Array with names
$a[] = "Anna";
$a[] = "Brittany";
$a[] = "Cinderella";
$a[] = "Diana";
$a[] = "Eva";
$a[] = "Fiona";
$a[] = "Gunda";
$a[] = "Hege";
$a[] = "Inga";
$a[] = "Johanna";
$a[] = "Kitty";
$a[] = "Linda";
$a[] = "Nina";
$a[] = "Ophelia";
$a[] = "Petunia";
$a[] = "Amanda";
```

```
$a[] = "Raquel";
$a[] = "Cindy";
$a[] = "Doris";
$a[] = "Eve";
$a[] = "Evita";
$a[] = "Sunniva";
$a[] = "Tove";
$a[] = "Unni";
$a[] = "Violet";
$a[] = "Liza";
$a[] = "Elizabeth";
$a[] = "Ellen";
$a[] = "Wenche";
$a[] = "Vicky";
```

PHP and AJAX

```
// get the q parameter from URL
$q = $_REQUEST["q"];

$hint = "";

// lookup all hints from array if $q is different from ""
if ($q !== "") {
    $q = strtolower($q);
    $len=strlen($q);
    foreach($a as $name) {
        if (stristr($q, substr($name, 0, $len))) {
            if ($hint === "") {
                $hint = $name;
            } else {
                $hint .= ", $name";
            }
        }
    }
}

// Output "no suggestion" if no hint was found or output correct values
echo $hint === "" ? "no suggestion" : $hint;
```

Email and PHP

- The mail() function allows you to send emails directly from a script.

Requirements

- For the mail functions to be available, PHP requires an installed and working email system.
- The program to be used is defined by the configuration settings in the php.ini file.

Installation

- The mail functions are part of the PHP core. There is no installation needed to use these functions.

Email and PHP

PHP Mail Functions

Function	Description
<u>ezmlm_hash()</u>	Calculates the hash value needed by EZMLM
<u>mail()</u>	Allows you to send emails directly from a script

Email and PHP

Name	Default	Description	Changeable
mail.add_x_header	"0"	Add X-PHP-Originating-Script that will include UID of the script followed by the filename. For PHP 5.3.0 and above	PHP_INI_PERDIR
mail.log	NULL	The path to a log file that will log all mail() calls. Log include full path of script, line number, To address and headers. For PHP 5.3.0 and above	PHP_INI_PERDIR
SMTP	"localhost"	Windows only: The DNS name or IP address of the SMTP server	PHP_INI_ALL
smtp_port	"25"	Windows only: The SMTP port number. For PHP 4.3.0 and above	PHP_INI_ALL
sendmail_from	NULL	Windows only: Specifies the "from" address to be used when sending mail from mail()	PHP_INI_ALL
sendmail_path	"/usr/sbin/sendmail -t -i"	Specifies where the sendmail program can be found. This directive works also under Windows. If set, SMTP, smtp_port and sendmail_from are ignored	PHP_INI_SYSTEM